

National University of Computer and Emerging Sciences

Lab Manual

Data Analysis and Visualization



Lab 04

Instructor

Rida Mahmood, Asif Ahsan

Class

BDS-5A/5D

Semester

Fall 2023

Fast School of Computing

FAST-NU, Lahore, Pakist

Objectives

★ Use OpenCV Library for Image Processing Tools

The objective of this lab is to introduce the student with OpenCV especially with respect to image processing.

References:

Lecture 6 and Lecture 7 of Theory

Theory:

Open Source Computer Vision Library (OpenCV) is an open source software library written in C++ for machine learning and computer vision applications. It is licensed under BSD licenses which make it suitable for use both in the industry and academia. Why OpenCV? OpenCV is a comprehensive collection of more than 2500 machine learning and computer vision algorithms that can be used from something as simple as detecting faces in images to project augmented reality overlaid with scenery. Another area in which OpenCV excels is its superior support for multiple interfaces and all the major operating systems as compared to other solutions. OpenCV supports Windows, Linux, OS X and Android and provides interfaces for C, C++, Python, Java and MATLAB. OpenCV is used extensively from tech giants such as Google, Microsoft, Intel, IBM etc. to start-ups like Applied Minds, VideoSurf and Zeitera. Some of the applications that can be accomplished easily with OpenCV are: identifying objects, tracking camera movements, stitching images together, finding similar images in a database using an image, face detection and tracking moving objects in a video feed etc.

Some Useful Commands:

1. To create a 2D array of zeros using NumPy: `my_array = numpy.zeros (( row, columns ))`
2. To create a 2D array of ones using NumPy: `my_array = numpy.ones (( row, columns ))`
3. To check the size of a 2D array: `size = numpy.shape(my_array)`
4. Reading an image using OpenCV: `my_image = cv2.imread("test_image.jpg",0)` The second argument determines whether the image is read as a grayscale image or a colored image. 0 is used for reading an image as grayscale and while 1 is used for reading in color.
5. Displaying an image using OpenCV: `cv2.imshow ("Title of the window", my_image)`. Two more commands that need to be used while displaying an image are: `cv2.waitKey(x)` `cv2.destroyAllWindows()`. The `waitKey()` function waits for a key being pressed for x number of milliseconds. If 0 is passed to `waitKey()` as an argument, it will wait indefinitely for a key press. `cv2.destroyAllWindows()` closes all the open image windows.
6. Writing an image to disk: `cv2.imwrite("image_name.jpg", my_image)`
7. Resizing an image: `cv2.resize(my_image, (new_height,new_width))`

LAB TASKS

Task 1: Convert RGB images to grayscale using the following formula. Use any RGB image of your choice, you can also use image given in the last question well.

grayscale image = $((0.3 * R) + (0.59 * G) + (0.11 * B))$

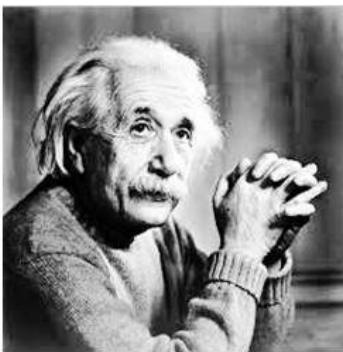
Task 2: Save data into JSON format.

Task 3: Make a function `make_histogram` which inputs the image and its size, and outputs the histogram of an image. Plot this histogram using matplotlib.

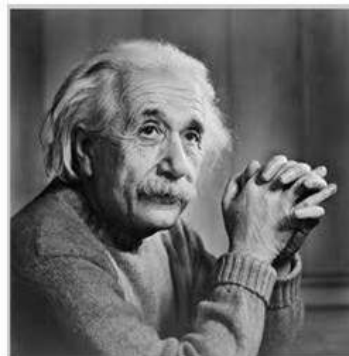
Image:

<https://i0.wp.com/www.themarginalian.org/wp-content/uploads/2013/05/einstein1.jpg?w=680&ssl=1>

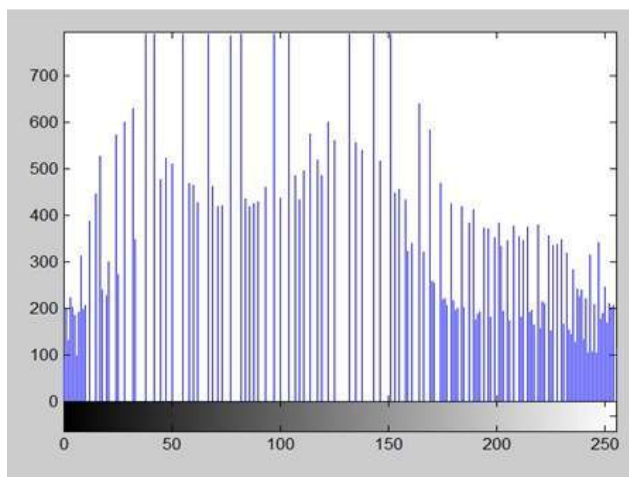
New Image



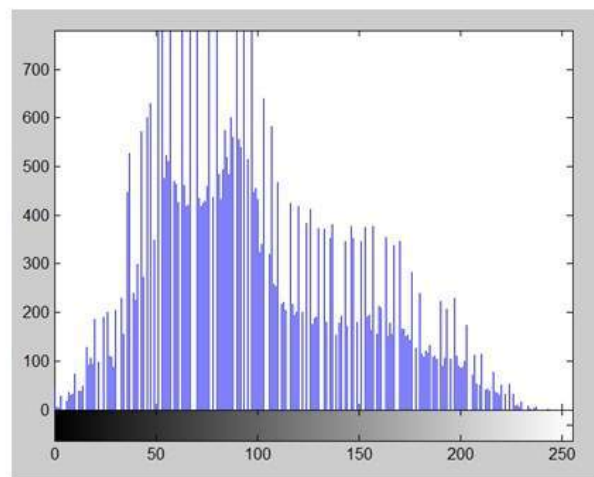
Old image



New Histogram



Old Histogram



Task 4:

Perform histogram normalization/equalization technique. You have to utilize the make_histogram function from previous task, need to write one more function of Commulative_sum , then you can calculate the equalized histogram by using given formula:

$$\text{CSD_histogram} = (\text{CD-min}) * 255 / (\text{max-min})$$

Task5:

Perform spatial filtering operation on following image, calculate its size, resize it to 16x16 and use kernel size 3x3.

Image:

https://upload.wikimedia.org/wikipedia/en/7/7d/Lenna_%28test_image%29.png

Kernel:

[3,0,3

1,0,1

3,0,3]